Class Diagram — Smart Home

ui

UserInterface
- current_user: User
- active_home_id: int

+ log_in(email, password)
+ log_out()
+ select_home(home_id)
+ control_device(device_id, action)
+ request_home(address, items)
+ request_device(room_id, desc)
+ get_status()
+ open_admin_panel()

«use» «use» «use» «use»

service

RequestController
- users: Repository[User]
- requests: Repository[Request]

+ request_home(user_id, address, items)
+ request_device(user_id, room_id, desc)

AdminController
- homes: Repository[Home]
- events: Repository[Event]
- requests: Repository[Request]

+ list_active_homes(): list[Home]
+ view_active_devices(home_id)
+ turn_on_system(home_id)
+ turn_off_system(home_id)
+ get_logs(): list[Event]
+ handle_request(request_id, approve: bool)

HomeController
- homes: Repository[Home]

+ control_device(home_id, device_id, action)
+ get_status(home_id): str

AutomationController
- homes: Repository[Home]
- events: Repository[Event]
- channel: CommunicationChannel

+ process_reading(device_id: int, value: str)
- evaluate_rules(h: Home, e: Event)

AuthService
- users: Repository[User]

+ log_in(email, password): User
+ log_out()
- verify_password(u, password): bool

NotificationService
- browser_permission: bool

+ request_permission()
+ send(n: Notification)

«use» «use» «use» «use» «use» «use» «use» «use»

receives readings

repository

«interface» Repository[T]
+ find(id: int): T
+ list_all(): list[T]
+ save(t: T)
+ delete(id: int)

PostgresRepository[T]
- table: str
- connection_url: str

+ find(id: int): T
+ list_all(): list[T]
+ save(t: T)
+ delete(id: int)

domain

Request
- id: int
- type: str
- description: str
- date: datetime
- status: str
- items: list[str]

+ submit()
+ approve(): Home
+ reject()

User
- id: int
- name: str
- email: str
- password_hash: str
- role: str (client | admin)

+ create_request(request_type, description, items): Request

Home
- id: int
- address: str
- system_status: str

+ turn_on_system()
+ turn_off_system()
+ get_status(): str
+ add_room(name)
+ find_device(device_id): Device
+ get_rules(): list[Rule]

Room
- id: int
- name: str

+ get_devices_in_use(): list[Device]
+ add_device(device_type, name)

«abstract» Device
- id: int
- name: str
- status: str

+ activate()
+ deactivate()
+ get_status(): str

NotifyAction
- message: str

+ execute()

Notification
- id: int
- message: str
- date: datetime
- is_read: bool

+ mark_as_read()
+ summary(): str

1 addressed to 0..*
0..* makes 1
1 owns 0..*
1 1..* «use» 1 0..*
«create»
0..1 (admin) handled by
0..1
0..* for 1
«use» send_action
0..*

«abstract» Sensor
- type: str

+ read_data(): float
+ generate_event(value): Event

Event
- id: int
- type: str
- value: str
- date: datetime

+ summary(): str

Rule
- id: int
- name: str
- is_active: bool

+ evaluate(e: Event): bool
+ execute()

«interface» Action
+ execute()

ActuatorAction
- command: str

+ execute()

«abstract» Actuator
+ execute_action(action: str)

1 generates 0..* 1 observes «use» 1 actions 1..* 1 target

condition 1

LightSensor
- light_level: float
- threshold: float

+ is_low_light(): bool

MotionSensor
- motion_detected: bool
- sensitivity: int

+ has_motion(): bool

0..* «interface» Condition
+ evaluate(e: Event): bool

ThresholdCondition
- operator: str
- value: float

+ evaluate(e: Event): bool

MotionCondition
+ evaluate(e: Event): bool

ServoMotor
- angle: int

+ move(angle: int)

LED
- color: str
- intensity: int

+ set_intensity(v: int)

Curtain
- opening_percentage: int

+ set_opening(p: int)

infrastructure

«interface» CommunicationChannel
+ connect()
+ send_action(device_id: int, action: str)
+ receive(): str

ServerClient
- server_url: str
- protocol: str
- network_port: int

+ connect()
+ send_action(device_id: int, action: str)
+ receive(): str
+ is_connected(): bool